#

```
import java.util.concurrent.atomic.AtomicLong;
public class AzharThread implements Runnable {
    static AtomicLong safeCounter = new AtomicLong(0);
    static long unsafeCounter = 0;
    long instanceCounter = 0;
    int incrementn;
    boolean threadsafe;

AzharThread(int incrementn, boolean threadsafe) {
    this.incrementn = incrementn;
    this.threadsafe = threadsafe;
}

@Override
public void run() {
    for (int i=0; i<incrementn; i++) {
        instanceCounter++;
        if (threadsafe) {
            safeCounter.incrementAndGet();
        }
    }
}
```

laxy A04s

CamScanner

```
        else {
            unsafeCounter++;
        }
    }
}

int numberofthreads;
int increments;
boolean threadsafe;
try {
numberofthreads = Integer.parseInt(args[0]);
increments = Integer.parseInt(args[1]);
} catch (NumberFormatException e) {
System.out.println("Thread count and
increments must be integers.");
    return;
}
```

axy A04s

CamScanner

```
threadSafe = Boolean.parseBoolean(args[2]);
safeCounter.set(0);
unsafeCounter=0;

for(int i=0; i<numberofThreads; i++){

tasks[i]=new AzharThread(increment,
                         threadSafe);

threads[i]=new Thread(tasks[i]);
threads[i].start();
}

for(int i=0; i<numberofThreads; i++){
   threads[i].join();
}
long nonStaticTotal = 0;
for(int i=0; i<numberofThreads; i++){
nonStaticTotal+=tasks[i].instanceCounter;
}
```

```
System.out.println("mode: "+(threadsafe ?
ThreadSAFE : "UNSYNCHRONIZED"));
System.out.println("threads: "+numberOfThreads);
System.out.println("Increments per threads:
    + increments);
System.out.println("Expected count: "+
    expectedCount);
System.out.println("static count: "+staticCo
System.out.println("non static Total:"+
    nonStaticTotal);
System.out.println("Absolute difference:"
    +difference);
if (Double.isNaN(percentageDifference)) {
System.out.println("percentage difference: unde
} else {
System.out.println("percentage difference: %.n
    ,percentageDifference);
}
}
```